keys, since it makes it easier to locate the data in the dictionary.

Update and delete functions are shown below. They are built using the same functions and constructs as the functions shown previously:

```
(defn update-person [connection person]
  (let [[params (, (:name person) (:number person) (:id
person))]]
    (try (with [connection] (.execute connection "update
person set name=?, phone=? where OID=?" params))
      (catch [e Exception] (print e)
             (print "failed to update person")))))

(defn delete-person [connection person-id]
  (try (with [connection] (.execute connection "delete from
person where OID=?" (, person-id)))
       (catch [e Exception] (print e)
           (print "failed to delete person"))))
```

User interface

The last step in our programming task is to tie everything together and write a user interface. I chose to write a text-based interface, since it is short and works on multiple platforms. Because the program supports both Python 2 and 3, we'll define a new function called key-input and, depending on the Python version, assign the input or raw input function into it. The 'if form' takes three parameters and the last one is optional. The first parameter is the test being performed. The second part is executed if the test evaluates *True;* otherwise, the last block is executed:

```
(if (= (get sys.version-info 0) 3) (def key-input input) (def
key-input raw-input))
```

Adding a person is a short function. First let's ask for the name and the phone number from the user and create a dictionary to represent a person. The dictionary is then passed to the insert-person function, which takes care of saving the person. The last step is to return *True* to inform the main loop that the user does not wish to quit yet:

```
(defn add-person [connection]
  (print "********************")
  (print "    add person")
  (print "")
  (let [[person-name (key-input "enter name: ")]
        [phone-number (key-input "enter phone number: ")]]
    (insert-person connection {:name person-name :number
phone-number :id None})
    True))
```

To display a person, let's write a function that simply takes a person dictionary and outputs it on screen. The display-person function is then used when the user wishes to search entries in the database.

To search for data from the database, we have the function, search-person. It asks the user for text, be it a name or phone number, and uses it to search from the database. Here we are using a new form—for. 'For' takes two parameters: an element, which is a collection pair in a form of lists or vectors, and a function to perform to each and every element. 'For' does not return anything, and results returned from the function are discarded:

```
(defn display-person [person]
  (print (:id person) (:name person) (:number person)))

(defn search-person [connection]
  (print "********************")
  (print "    search person")
  (print "")
  (let [[search-criteria (key-input "enter name or phone
number: ")]]
    (for (person (query-person connection search-criteria))
(display-person person)))
  True)
```

Editing a person is a somewhat more involved function. First, we load a person using an ID number that the user gives. If the ID is incorrect and not found in the database, the program notifies the user and returns to the main menu. However, if the person is actually found, the name and phone number are shown on screen. After this, the user can update the data and save it into the database again.

The code takes advantage of truth values. Python and Hy, in turn, have a convention where 'None', 'False', 'zero' and an empty sequence or mapping are considered false, while any other value is considered true. The function tries to load the person by a given ID and only if it is found, continues into editing. Likewise, when a user simply presses 'Enter' to give a name or phone number, the code detects this and uses the old value instead:

```
(defn edit-person [connection]
  (print "********************")
  (print "     edit person")
  (print "")
  (let [[person-id (key-input "enter id of person to edit:
")]
    [person (load-person connection person-id)]]
    (if person (do (print "found person")
          (display-person person)
          (let [[new-name (key-input "enter new name or press
enter: ")]
                [new-number (key-input "enter new phone or press
enter: ")]
                [edited-person {:id (:id person)
                       :name (if new-name new-name (:name
person))
                       :number (if new-number new-number
```